

REPORT FOR BINANCE USDT-M FUTURES TRADING BOT

1. Introduction

This project implements a command-line trading bot for Binance USDT-M Futures using Python. The goal of this assignment is to design a structured, modular bot capable of placing multiple types of Futures orders using the Binance API. The bot supports:

- Market Orders
- Limit Orders
- Stop-Limit Orders
- OCO Orders
- TWAP Strategy
- Grid Strategy

All trading is performed against the Binance USDT-M Futures Testnet, so no real funds are used. The bot also implements input validation, error handling, and a logging system that records activity in bot.log. This report explains the design, architecture, order types, and execution behaviour of the bot.

2. Project Architecture

The project follows a clean modular structure with separate Python files organized under src/. Each file handles a specific responsibility such as configuration, validation, client creation, or order execution. The overall folder structure is shown below.

```
kiran_binance_bot/
├── .venv/
├── .env
├── bot.log
├── src/
│   ├── config.py
│   ├── utils.py
│   ├── client.py
│   ├── market_orders.py
│   ├── limit_orders.py
│   └── advanced/
│       ├── stop_limit.py
│       ├── oco.py
│       ├── twap.py
│       └── grid.py
├── README.md
└── report.pdf
```

3. Key Components Explained

3.1 config.py – Configuration & Credentials

- Loads API key and secret from .env
- Enables testnet mode using BINANCE_TESTNET=true
- ensure_credentials() stops execution if credentials are missing

3.2 utils.py – Validation & Logging

Validation helpers:

- validate_symbol(symbol)
- validate_side(side)
- validate_positive_float(value)
- validate_price_hierarchy(stop_price, limit_price)

Logging:

- Writes structured messages to bot.log
- Format: YYYY-MM-DD HH:MM:SS | module_name | LEVEL | message

3.3 client.py – Binance Futures Client

- Uses the python-binance library
- Configured to talk to Binance USDT-M Futures Testnet
- Shared by all order modules

3.4 market_orders.py – Market Orders

- Parses CLI arguments (symbol, side, quantity)
- Validates inputs using utils.py
- Sends MARKET order requests to the Futures API
- Logs every attempt and error

3.5 limit_orders.py – Limit Orders

- Accepts symbol, side, quantity and limit price
- Uses GTC (Good-Till-Cancelled) time in force
- Logs successful order placements and failures

4. Advanced Order Strategies

4.1 Stop-Limit Orders

Allows the user to define a stop trigger and a limit execution price.

Example command:

```
python src/advanced/stop_limit.py BTCUSDT BUY 0.001 65000 64950
```

4.2 OCO Orders (Simulated)

Places a logical One-Cancels-the-Other pair:

- A take-profit LIMIT order
- A stop-loss STOP-LIMIT order

4.3 TWAP Strategy

TWAP (Time-Weighted Average Price) splits a large order into equal slices over time.

Example:

```
python src/advanced/twap.py BTCUSDT BUY 0.1 5 10
```

4.4 Grid Strategy

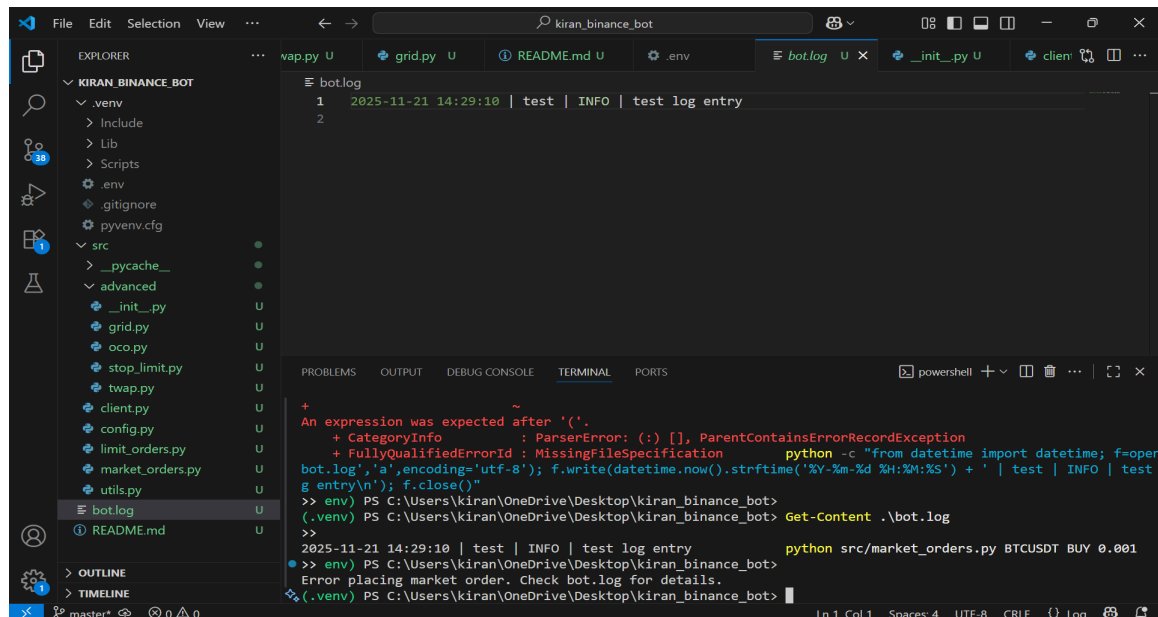
Places multiple limit orders between two price levels, creating a simple grid.

Example:

```
python src/advanced/grid.py BTCUSDT BUY 0.05 60000 70000 5
```

5. Execution Screenshots

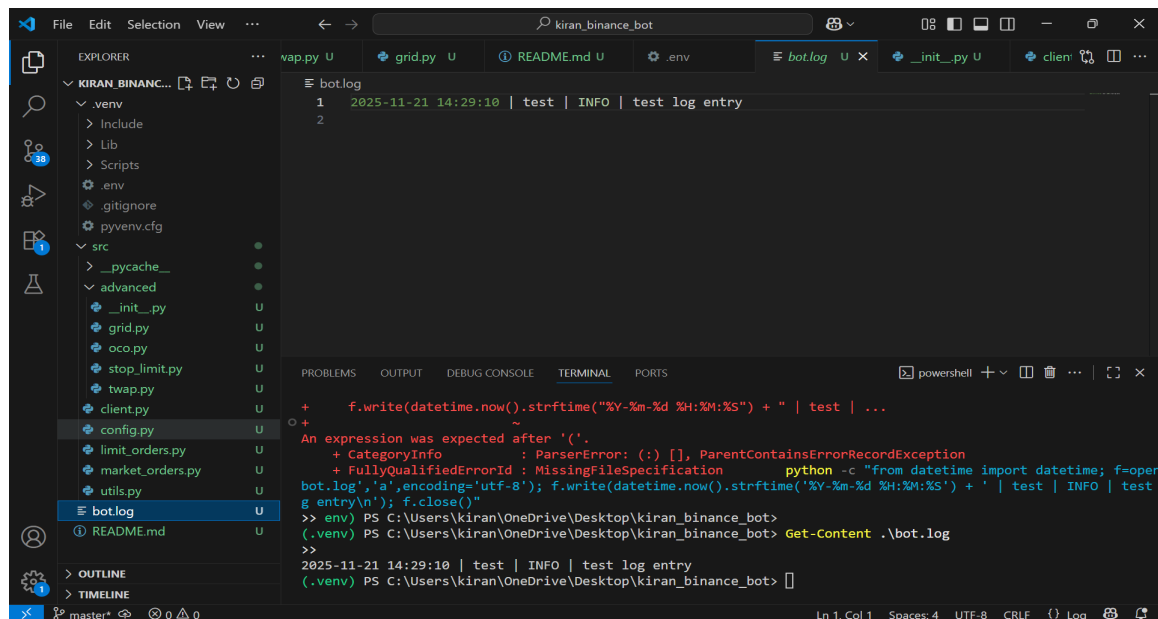
Screenshot 1: Terminal output when executing a sample MARKET order.



```
bot.log
1 2025-11-21 14:29:10 | test | INFO | test log entry
2

+
+ CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
+ FullyQualifiedErrorId : MissingFileSpecification python -c "from datetime import datetime; f=open
bot.log','a',encoding='utf-8'); f.write(datetime.now().strftime('%Y-%m-%d %H:%M:%S') + ' | test | INFO | test
g entry\n'); f.close()"
>> env) PS C:\Users\kiran\OneDrive\Desktop\kiran_binance_bot>
(.venv) PS C:\Users\kiran\OneDrive\Desktop\kiran_binance_bot> Get-Content .\bot.log
>>
2025-11-21 14:29:10 | test | INFO | test log entry python src/market_orders.py BTCUSDT BUY 0.001
>> env) PS C:\Users\kiran\OneDrive\Desktop\kiran_binance_bot>
Error placing market order. Check bot.log for details.
(.venv) PS C:\Users\kiran\OneDrive\Desktop\kiran_binance_bot>
```

Screenshot 2: Contents of bot.log showing a test log entry.



```
bot.log
1 2025-11-21 14:29:10 | test | INFO | test log entry
2

+
+ CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
+ FullyQualifiedErrorId : MissingFileSpecification python -c "from datetime import datetime; f=open
bot.log','a',encoding='utf-8'); f.write(datetime.now().strftime('%Y-%m-%d %H:%M:%S') + ' | test | INFO | test
g entry\n'); f.close()"
>> env) PS C:\Users\kiran\OneDrive\Desktop\kiran_binance_bot>
(.venv) PS C:\Users\kiran\OneDrive\Desktop\kiran_binance_bot> Get-Content .\bot.log
>>
2025-11-21 14:29:10 | test | INFO | test log entry
(.venv) PS C:\Users\kiran\OneDrive\Desktop\kiran_binance_bot>
```

6. Error Handling

The bot handles a variety of error conditions safely:

- Missing or invalid API keys
- Invalid trading pairs
- Non-positive quantity or price values
- Inconsistent stop and limit price relationships

- Exceptions returned by the Binance API during order placement

When errors occur, a human-readable message is printed to the terminal and full details are recorded in bot.log for later debugging.

7. Conclusion

This project successfully demonstrates a working Binance USDT-M Futures trading bot with:

- Clean, modular Python architecture
- Ability to connect to Binance Futures Testnet
- Support for Market, Limit, Stop-Limit, OCO, TWAP and Grid orders
- Robust input validation and error handling
- A logging system that records activity to bot.log
- CLI-based interaction suitable for automation or further extension

The design is ready for evaluation and can be extended with additional strategies or risk-management modules in future work.